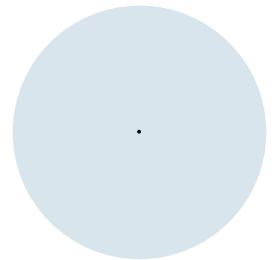


Arrays



Arrays

- Zur Motivation: Ein Kalender
- Viele wiederkehrende Fälle
- Bsp:
 - Terminverwaltung
 - Zeiterfassung

Montag											
00:00	01:00	02:00	03:00	04:00	05:00	06:00	07:00	08:00	09:00	10:00	11:00
						Wecker klingelt			Jetzt wirklich aufstehen		Vorlesung
12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00	21:00	22:00	23:00
	Mittagessen				Fitness studio		Pizza bestellen			Zocken	

Umsetzung (umständlich)

- Vereinfacht: Nur ein Wochentag

```
void main() {  
    java.util.Scanner scanner =  
        new java.util.Scanner(System.in);  
    // Für jede Stunde eine neue Variable  
    String appointment00 = "";  
    String appointment01 = "";  
    ...  
    String appointment22 = "";  
    String appointment23 = "";  
    ...  
}
```



code08.
Complicated
Calendar

Umsetzung (umständlich)

- Vereinfacht: Nur ein Wochentag

```
...
    String appointment23 = "";
    // Hauptschleife
    boolean done = false;
    while (!done) {
        // Auswahlmenü
        System.out.println("1 = Neuer Eintrag");
        System.out.println("2 = Termine ausgeben");
        System.out.println("3 = Beenden");
        int choice = scanner.nextInt();
        // Fallunterscheidung
        switch (choice) {
            ...
        }
    }
}
```



code08.
Complicated
Calendar

Umsetzung (umständlich)

```
switch (choice) {
    case 1 -> { // Neuer Eintrag
        System.out.print("Wieviel Uhr? ");
        int hourInput = scanner.nextInt();
        System.out.print("Termin? ");
        String appointment = scanner.next();
        switch (hourInput) {
            case 0 -> appointment00 = appointment;
            case 1 -> appointment01 = appointment;
            ...
            case 23 -> appointment23 = appointment;
        }
    }
    :
}
}
```



code08.
Complicated
Calendar

Umsetzung (umständlich)

```
case 2 -> { // Termine ausgeben
    System.out.println(" 0h: " +
        appointment00);
    System.out.println(" 1h: " +
        appointment01);

    ...
    System.out.println("23h: " +
        appointment23);
    System.out.println("-----");
}
:
}
}
}
}
```



code08.
Complicated
Calendar

Umsetzung (umständlich)

```
case 3 -> // Beenden
    done = true;
default -> // Falsche Eingabe
    System.out.println("Eingabefehler");
```

```
}
```

```
}
```

```
}
```

```
}
```



code08.
Complicated
Calendar

Umsetzung (umständlich)

- In Summe 108 Zeilen
- Viele ähnliche Zeilen
 - 24x Variablendeklaration
 - 24 Fallunterscheidungen bei Termineingabe
 - 24x Ausgabe einzelner Termine
- Jahreskalender: $365 \cdot 24 \cdot 3 = 26.280$ Zeilen Code
- Geht das nicht einfacher?



code08.
Complicated
Calendar

Arrays

- engl. für "Feld", "Bereich"
- **Zusammenfassung** mehrerer Variablen gleichen Typs
- Unterscheidung durch Index
- Einzelne Array"variablen" heißen Elemente
 - `appointments[0]` (statt `appointment00`)
 - `appointments[1]` (statt `appointment01`)
 - ...
 - `appointments[23]` (statt `appointment23`)

Index für Elementzugriff

- Alle Elemente eines Arrays vom **gleichen Typ**
- **Index** vom Typ `int`
 - **Erstes** Element mit **Index 0**
 - Nur **positive** Werte für Index
 - Index bestimmt **Position** eines Elements innerhalb des Arrays
- Array ~ eindimensionale Tabelle

Eindimensionale Tabelle

– appointments

0	1	2	3	4	5	6
"Zzzz"	" "	" "	" "	" "	" "	"ring"

- appointments[0] == "Zzzz";
- appointments[1] == " ";
- appointments[2] == " ";
- appointments[3] == " ";
- appointments[4] == " ";
- appointments[5] == " ";
- appointments[6] == "ring"

Arrays

- Zur Motivation: Ein Kalender
- Viele wiederkehrende Fälle
- Bsp:
 - Terminverwaltung
 - Zeiterfassung

Montag											
00:00	01:00	02:00	03:00	04:00	05:00	06:00	07:00	08:00	09:00	10:00	11:00
						Wecker klingelt			jetzt wirklich aufsteh en		Vorlesu ng
Mittwoch											
12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00	21:00	22:00	23:00
Mittag essen					Fitness studio		Pizza bestelle n				Zocken

Deklaration

```
int[] numbers;  
String[] names;  
  
// Erzeugt int-Array mit Elementen 0..41  
numbers = new int[42];  
  
// Erzeugt String-Array mit Elementen 0..1  
names = new String[2];
```

```
// Oder in einem Schritt (Erzeugt double-Array mit  
// Elementen 0..499)  
double[] temperatures = new double[500];
```

- Vor Erzeugung mit new kann nicht auf Array-Elemente zugegriffen werden

Initiale Werte

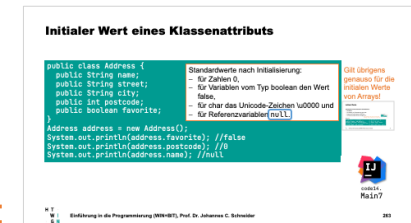
Standardwerte von Array-Elementen nach Initialisierung:

- für Zahlen 0,
- für Variablen vom Typ boolean den Wert false,
- für char das Unicode-Zeichen `\u0000` und
- für Referenzvariablen (Enums, Strings) `null`.

`null` bedeutet so viel wie "noch kein Inhalt hinterlegt". Mehr zum Thema `null` in Kapitel 13:

Beispiele:

```
int[] numbers = new int[6];           // {0, 0, 0, 0, 0, 0}
double[] grades = new double[3];      // {0.0, 0.0, 0.0}
boolean[] switches = new boolean[3];  // {false, false, false}
DayOfWeek[] weekend = new DayOfWeek[2]; // {null, null};
String[] names = new String[4];       // {null, null, null, null};
```



Vereinfachung Kalender (Live-Coding)

- Wir ersetzen im `ComplicatedCalendar` die 24 String-Variablen durch ein Array vom Typ `String` mit 24 Einträgen
- Fahrplan:
 - 24 Variablen entfernen und Array-Variable einführen
 - Array-Variable mit leeren Strings initialisieren (1. Version, umständlich)
 - Termine einlesen (`hourInput`)
 - Array-Variable mit leeren Strings initialisieren (2. Version, elegant mit Schleife)
 - Ausgabe der Termine
 - "Magische Zahl" 24 entfernen und erst durch Konstante, dann durch Array-Länge ersetzen
 - Nur gültige Stunden-Eingabe zulassen
- Das alles folgt (zur Nachbereitung) auch noch mal auf den folgenden Folien als Einzelschritte (und verschiedene Versionen von `SimpleCalendar1`, `SimpleCalendar2`, ...)

Vereinfachung Kalender

```
public class SimplerCalendar{  
  
    public static void main(String[] args) {  
        java.util.Scanner scanner =  
            new java.util.Scanner(System.in);  
        // Für alle Stunden zusammen ein Array  
        String[] appointments = new String[24];  
        ...  
    }  
}
```

- `Typ[] Variablenname; bzw.`
`Typ[] Variablenname = new Typ[Arraygröße];`



code08.
Simpler
Calendar1

Wertzuweisungen

- Einzelwerte
 - Variablenname[Index] = Wert;

```
appointments[0] = "Schlafen";  
appointments[1] = "";  
appointments[2] = "";  
appointments[3] = "";  
appointments[4] = "";  
appointments[5] = "";  
appointments[6] = "";  
// ...  
appointments[22] = "";  
appointments[23] = "";
```



code08.
Simpler
Calendar2a

Wertzuweisungen

- Einzelwerte
 - Variablenname[Index] = Wert;

```
case 1: // Neuer Eintrag
    System.out.print("Wieviel Uhr? ");
    int hourInput = scanner.nextInt();
    System.out.print("Termin? ");
    appointments[hourInput] = scanner.next();
    break;
```



code08.
Simpler
Calendar2a

Wertzuweisungen

- Mehrere Werte in Folge
 - **Schleife**, z.B. für Belegung mit leerem String

```
String appointments[] = new String[24];  
for (int hour = 0; hour < 24; hour++) {  
    appointments[hour] = "";  
}
```



code08.
Simpler
Calendar2b

Elementzugriff

- Über Variablenname[Index]

```
System.out.println(appointments[12]);
```

- Index kann auch Wert einer `int`-Variable sein, z.B. einer Schleifenvariable:

```
String appointments[] = new String[24];  
// ...  
for (int hour = 0; hour < 24; hour++) {  
    System.out.println(hour + ": " + appointments[hour]);  
}
```



code08.
Simpler
Calendar3

Vereinfachung Kalender

- Einsparung 113 → 48 Zeilen Quellcode bei gleicher Funktionalität
 - Bessere Wartbarkeit des Codes
 - Bessere Erweiterbarkeit (z.B. mehr als 24 Einträge pro Tag)

Arraygröße

- Im Kalenderbeispiel "hartcodiert" als Wert 24 ("magic number")
- Bei Änderung (z.B. man will Termine halbstündlich speichern -> neue Größe 48) überall im Code nach 24 suchen → Fehleranfällig
- Lösung 1: Länge in Konstante speichern (`final` → unveränderlich)

```
final int NR_OF_APPOINTMENTS = 24;
String[] appointments =
    new String[NR_OF_APPOINTMENTS];
:
for (int hour = 0; hour < NR_OF_APPOINTMENTS; hour++) {
    ...
}
```



code08.
Simpler
Calendar4

Arraygröße

- Lösung 2: Auf Länge über Arrayvariablenname.length zugreifen

```
for (int hour = 0; hour < appointments.length; hour++) {  
    ...  
}
```

- Gut, denn man hat alle Informationen über das Array aus dem Array selbst (Länge und Inhalt)
 - spart Extra-Variable für die Länge
 - später, wenn wir Arrays als Parameter an Methoden übergeben, reicht das Array aus (man muss nicht zusätzlich noch die Länge mitgeben)



code08.
Simpler
Calendar5

Arraygröße

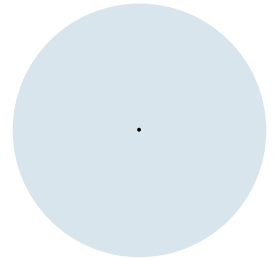
- `Arrayvariablenname.length`: Hilfreich, um Zugriffsfehler zu vermeiden (`ArrayIndexOutOfBoundsException`)

```
int hourInput = scanner.nextInt();
if (hourInput < 0 || hourInput >= appointments.length) {
    System.out.println("keine gültige Stunde");
    break;
}
```



code08.
Simpler
Calendar6

for-each-Schleife



Schleifen für Arrayelemente

- Schleife mit explizitem Aufzählen

```
int[] primeNumbers = {2, 3, 5, 7, 11};  
for (int i = 0; i < primeNumbers.length; i++) {  
    System.out.println(primeNumbers[i]);  
}
```

- for-each-Schleife

```
for (int primeNumber : primeNumbers) {  
    System.out.println(primeNumber);  
}
```

➔ Jedes Element wird der Reihenfolge nach genau einmal durchlaufen



for-each-Schleife

```
for (<Elementtyp> <Variable> : <Array>) {  
    // verwende <Variable>  
}
```

- <Array>.length Durchläufe
- Bei jedem Durchlauf Zuweisung des nächsten Elements an <Variable>
- Verwendung von <Variable> im Schleifenrumpf

➔ Schließt Fehler bei Aufzählung aus, z.B.

```
for (int i = 1; i < primeNumbers.length; i++) { ... }  
for (int i = 0; i <= primeNumbers.length; i++) { ... }  
for (int i = 0; i < primeNumbers.length - 1; i++) { ... }
```

Wann die for-each-Schleife verwenden?

- **IMMER!!!** (wenn es geht)
- Von jetzt an nie mehr mit `for (int i = ...) über Array iterieren!`
- außer:
 - Man braucht den Wert des Indexes eines Arrayeintrags innerhalb der Schleife und/oder
 - man will Array-Einträge verändern

Mit for-each-Schleife nicht möglich (1)

- Zugriff auf Index des Array-Eintrags

```
int[] primeNumbers = {2, 3, 5, 7, 11};  
  
for (int i = 0; i < primeNumbers.length; i++) {  
    System.out.println("Die " + (i + 1)  
        + "-te Primzahl ist " + primeNumbers[i]);  
}
```



code08.

ForEach
NotPossible1

Mit for-each-Schleife nicht möglich (2)

- Ändern von Array-Einträgen

```
int[] numbersToDouble = {1, 2, 3, 4};  
  
for (int i = 0; i < numbersToDouble.length; i++) {  
    numbersToDouble[i] *= 2;  
}
```

Hier wird der i-te Eintrag geändert

```
int[] numbersToDouble = {1, 2, 3, 4};  
  
for (int numberToDouble: numbersToDouble) {  
    numberToDouble *= 2;  
}
```

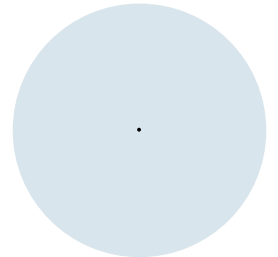
Hat keine Auswirkungen auf das Array



code08.

ForEach
NotPossible2

Arrays – Nützliches und Wissenswertes



Deklaration + Erzeugung + Init

- Auch alles in einem Schritt möglich:

```
int[]    numbers    = {2, 3, 5, 7, 11, 13};  
double[] values     = {3.1415926535, 2.71828};  
String[] someWords  = {"Dieser", "Satz", "kein", "Verb"};
```

- {}-Ausdruck heißt Arrayinitialisierer
- Arraygröße ergibt sich aus Anzahl der Werte im Arrayinitialisierer
- Geeignet, wenn alle Werte bekannt sind (insbesondere bei wenigen Werten) und keine Schleife sinnvoll ist

Arrayinhalt auf der Konsole ausgeben

- Keine brauchbare Möglichkeit: direkt ausgeben

```
int[] array = {17, 23, 42};  
System.out.println(array);  
// Ausgabe wie [I@7e0ea639, ähnlich der "Speicheradresse"
```

- Gute, flexible Möglichkeit: Ausgabe mit Schleife

```
for (int i = 0; i < array.length; i++) {  
    System.out.println(i + ": " + array[i]);  
}
```

- Praktisch zum print-Debuggen: `Arrays.toString`

```
System.out.println(Arrays.toString(array));  
// Ausgabe [17, 23, 42]
```

jshell>



code08.
ArrayTo
Console

Aufpassen: .length vs. max. Index

- Auf ein Array der Länge **n** greift man über die Indizes 0, 1, ..., **n-1** zu!
- Maximaler Index eines Arrays array ist `array.length - 1`!
- Beispiel: `numbers` hat die Länge 6. Man greift auf die Elemente über den Index 0 bis 5 zu.

```
//      Index:   0  1  2  3  4  5
int[] numbers = {2, 3, 5, 7, 9, 11};
System.out.println(numbers.length);
// Ausgabe: 6
```

- Zugriff auf das letzte Element eines Arrays:

```
int lastNumber = numbers[numbers.length - 1];
```

- Ähnlich bei String, vgl. Code in `code05.StringMethods`



code08.
LengthVs
Index

Arrayinitialisierer nachträglich

- Arrayinitialisierer:

```
int[] numbers = {2, 3, 5, 7, 11, 13};
```

- Nachträglich nicht möglich:

```
int[] numbers = {2, 3, 5, 7, 11, 13};  
numbers = {1, 2, 3, 4, 5}; // nicht möglich
```

- Hier schreibt man new <Typ>[] { Werte }

```
int[] numbers = {2, 3, 5, 7, 11, 13};  
numbers = new int[]{1, 2, 3, 4, 5};
```

Array muss initialisiert sein

- Variable wird deklariert, aber nicht initialisiert → Compiler-Fehler!

```
String[] appointments;  
  
//Compiler-Fehler: not initialized  
appointments[0] = "Treffen";
```

- In `ArrayInitialization` auch Zusammenfassung von drei möglichen Methoden zur Array-Initialisierung



code08.
ArrayIni-
tialization

Arrayinhalt abhängig vom Index

- Beispiel Wertetabelle (array[x] = f(x) für mathematische Funktion)

```
Scanner scanner = new Scanner(System.in);
System.out.print("Array size? ");
int size = scanner.nextInt();
int[] squareNumbers = new int[size];
for (int x = 0; x < squareNumbers.length; x++) {
    squareNumbers[x] = x * x;
}
```

i	0	1	2	3	4	5	6
squareNumbers[i]	0	1	4	9	16	25	36



code08.
Square
Numbers